

AULA PRÁTICA · 45 MIN

Como montar seus agentes, passo a passo

Da pasta vazia até um pipeline rodando. Quinze passos, os arquivos exatos, e os erros que vocês vão cometer.

Nasser · Claude Code e Antigravity

O que você vai ter no fim desta aula

Uma pasta montada

Seis pastas, cada uma com uma regra clara de quem escreve nela.

Três agentes escritos

Um pesquisador, um consolidador e um revisor — os papéis que resolvem 90% dos casos.

Um pipeline que roda

Pesquisa em paralelo, consolidação, e revisão com loop de correção.

Serve para qualquer trabalho com a forma “**pesquisar várias coisas → juntar → conferir**” — que é a forma da maioria dos trabalhos de faculdade e de consultoria.

Uso o meu projeto só como exemplo de cada arquivo. Vocês trocam o assunto e a estrutura continua a mesma.

O que é um subagente

Uma instância separada do modelo, que recebe **uma** tarefa, tem contexto e ferramentas próprios, escreve um arquivo e morre.

Contexto próprio

Não herda a sua conversa.
Começa limpo.

Ferramentas próprias

Só o que você declarar no
arquivo dele.

Uma saída

Um arquivo, formato fixo,
caminho definido por você.

Efêmero

Some no fim. O que sobra é
o arquivo.

Na prática: **um subagente é um arquivo markdown**. Cabeçalho com nome, ferramentas e modelo; o prompt dele embaixo.

O CAMINHO

Quinze passos

a base

- 1 criar as pastas
- 2 a constituição
- 3 as permissões
- 4 cabeçalho do agente
- 5 corpo do agente

o contrato

- 6 template de saída
- 7 as ferramentas
- 8 o modelo
- 9 nomes de arquivo
- 10 o pesquisador

rodar

- 11 o consolidador
- 12 o revisor
- 13 as três fases
- 14 loop de correção
- 15 quando virar skill

PASSO 1

Crie a estrutura de pastas

```
meu-projeto/
├── CLAUDE.md           a constituição: todo agente lê
├── .claude/
│   ├── settings.json  permissões do Bash
│   ├── agents/        um .md por agente
│   └── skills/        receitas (passo 15)
├── input/             o que você RECEBEU
├── pdf/               fontes que você BAIXOU
├── scripts/           ferramentas suas
└── output/            tudo que os agentes ESCRIVEM
```

Três procedências diferentes: **entrada dada** (input), **entrada coletada** (pdf), **saída gerada** (output). Misturou, em duas semanas ninguém sabe mais o que era origem e o que o agente escreveu.

Quem escreve em cada pasta

Pasta	Quem escreve	Regra
input/	só você	enunciado, material do cliente, insumos dos colegas — agente só lê
pdf/	só você	o que o agente não baixou, você põe aqui. É fonte, não cache
scripts/	só você	agente executa, nunca edita
output/	só agente	um arquivo por agente, nome fixo definido no passo 9
.claude/	só você	nome do arquivo de agente = nome do agente, sem o .md

Precisa transformar um arquivo de input/? O resultado vai para output/. O original fica intacto — é a única forma de refazer tudo do zero quando algo der errado.

Escreva a constituição — CLAUDE.md

Um arquivo na raiz que **todo** agente lê automaticamente. Se você repetir a regra dentro de cada prompt, no dia que mudar você muda em seis lugares e esquece um.

01 Projeto e objetivo

Três linhas. O que estamos fazendo e para quê.

02 Escopo

O que é meu, o que é dos colegas.

03 Regras de diretório

A tabela do slide anterior, escrita.

04 Regras de rigor

Citação, níveis de fonte, e o que fazer quando não achar.

05 O pipeline

Quem roda quando, e o que cada um escreve.

06 Glossário

As siglas do seu domínio. Evita o agente inventar termo.

As duas seções que mais importam

regras de diretório

```
## Regras de diretório
- ESCRITA: exclusivamente em ./output/
- LEITURA: livre no repositório
- ./input/ e ./pdf/ são só leitura
- Downloads vão para pasta temporária
  do sistema, nunca para dentro daqui
```

regras de rigor

```
## Regras de rigor
1. Toda afirmação carrega fonte.
2. Citação: [n] no texto + ## Fontes
3. Níveis: PRIMÁRIA / IMPRENSA /
  RELAT&Oacute;RIO / FABRICANTE
4. Lacuna é resultado válido.
```

A regra 4 é a mais importante do arquivo inteiro. Sem ela, o agente encurralado **inventa** — entregar alguma coisa parece melhor que entregar nada.

Trave as permissões — .claude/settings.json

```
{  
  "permissions": {  
    "allow": [ "Bash(python3 ./scripts/baixar_pdf.py:*)" ],  
    "deny": [ "Bash(rm:*)" , "Bash(mv:*)" , "Bash(curl:*)" ,  
              "Bash(wget:*)" , "Bash(sudo:*)" , "Bash(git push:*)" ]  
  }  
}
```

A regra do passo 2 diz “só escreva em output/”. Isso é um **pedido**, não uma trava — um agente com Bash pode rodar rm mesmo assim. Este arquivo é a trava: o deny bloqueia antes de qualquer aprovação. O allow faz o contrário: libera o único comando que você quer, sem aprovar na mão toda vez.

O cabeçalho do agente

```
---
name: pesquisador-editais
description: Levanta requisitos
            técnicos de editais. Use para...
tools: WebSearch, WebFetch,
       Read, Write, Glob, Grep
model: sonnet
color: orange
---
```

name — igual ao nome do arquivo, sem o .md.

description — **quando** usar este agente. É o que faz o agente principal escolher ele.

tools — passo 7. O que não estiver aqui, ele não tem.

model — passo 8.

color — só a cor na interface. Ajuda a achar quem é quem quando cinco rodam juntos.

Erro clássico: description dizendo o que o agente é em vez de *quando* chamar. “Especialista em editais” não ajuda ninguém a decidir nada.

O corpo do agente — sempre as mesmas 5 seções

01 A pergunta única

Uma frase, logo no início. Se não couber numa frase, são dois agentes.

02 Pré-condição

O que ler antes — a constituição, os arquivos de input/ — e o que fazer se faltar: **pare e reporte**.

03 Método

O passo a passo da busca. Onde procurar primeiro, o que priorizar, o que ignorar.

04 Regras de rigor específicas

O que ele **não** pode fazer. É a seção que mais evita alucinação.

05 Template de saída

O arquivo exato que ele vai escrever. A mais importante das cinco — tem o passo 6 inteiro só para ela.

O template de saída – as cinco regras

01 Cole o esqueleto inteiro no prompt.

Com os ## já escritos. Não *descreva* o formato — *mostre* o arquivo pronto, vazio.

02 Toda tabela tem as colunas nomeadas.

“Faça uma tabela comparativa” devolve cinco tabelas diferentes em cinco rodadas.

03 Duas seções obrigatórias em todo output.

Lacunas de verificação e ## Fontes. Sem exceção.

04 Comece pelo resumo, com limite de tamanho.

“## Resumo executivo (10 linhas)”. Sem o limite vem uma página.

05 Diga o caminho exato, na última linha.

É o contrato do passo 9 se materializando.

O esqueleto, anotado

```
Escreva em ./output/05-editais.md:

# Editais - especificação e preço

## Resumo executivo ← 10 linhas

## Achados
### <nome do achado>
- Constatação [n]
- Trecho decisivo (máx. 15 palavras) [n]

## Tratamento do tema ← colunas FIXAS
| Documento | O que fixa | Vincula? | Fonte |

## Lacunas de verificação
| O que falta | O que tentei | Prioridade |

## Fontes
1. [PRIMÁRIA] Título - órgão - URL - data
```

O [n] amarra cada afirmação à sua fonte. Sem ele, revisar é impossível.

A tabela de lacunas tem a coluna “**o que tentei**” de propósito. Lacuna sem isso é desculpa; com isso, é resultado.

O output muda conforme o papel

Pesquisador

`output/05-editais.md`

Resumo executivo

Achados

Tabela do tema

Lacunas

Fontes

"o que eu encontrei?"

Consolidador

`output/00-consolidado.md`

Resumo executivo

Tabela unificada

Divergências

Lacunas consolidadas

Fontes renumeradas

"o que o conjunto diz?"

Revisor

`output/00-revisao.md`

PASS: X/5 · FAIL: Y/5

ENCAMINHAMENTO:

Detalhe por item

Itens que falharam

"posso seguir?"

O revisor é o único cujo output **não é conteúdo** — **é uma decisão**. Por isso começa pelo placar e pela linha de encaminhamento, não por prosa.

Escolha as ferramentas — por papel

Papel	Dê	Nunca dê
Pesquisador	WebSearch, WebFetch, Read, Write, Glob	—
Consolidador	Read, Write, Glob, Grep	WebSearch, WebFetch
Revisor	Read, Glob, Grep	WebSearch, WebFetch
Analista	Read, Write	WebSearch, Bash

Por que tirar a busca do revisor: ao encontrar uma lacuna, um revisor com WebSearch vai **pesquisar e preencher** — e aí deixou de ser revisor. Sem a ferramenta, a única saída dele é reprovar.

Escolha o modelo

sonnet — critério fechado

Ou o dado está na fonte, ou não está. Pesquisadores, revisor mecânico, gerador de PDF. É a maioria dos seus agentes.

opus — ambiguidade e risco

Onde se decide o que entra e o que vira lacuna. Consolidador, analista, e qualquer agente cujo assunto tenha pouco dado público.

Regra prática: **se o agente pode errar para o lado de inventar, é opus**. Se o pior erro dele é não achar, é sonnet.

Defina os nomes de arquivo *antes* dos prompts

```
output/01-<tema>.md      ← pesquisador 1
output/02-<tema>.md      ← pesquisador 2
output/03-<tema>.md      ← pesquisador 3
output/00-consolidado.md ← consolidador
output/00-revisao.md     ← revisor
```

Os agentes não conversam entre si. Eles se encontram no sistema de arquivos — por isso o nome do arquivo é contrato, e você define ele antes de escrever qualquer prompt.

Se o número de pesquisadores for variável, fixe um **padrão de slug** e obrigue no prompt. O consolidador acha todos por glob; se o slug variar, o arquivo **some da consolidação sem dar erro nenhum**.

PASSO 10

Escreva o pesquisador

```
---
name: pesquisador-editais
tools: WebSearch, WebFetch, Read, Write
model: sonnet
---

# 1. pergunta única
como os editais tratam o tema?

# 2. pré-condição
Leia ./CLAUDE.md e ./input/enunciado.md.
Verifique ./pdf/ ANTES da web.

# 3. método - órgão, últimos 2 anos.

# 4. rigor
Não decida sozinho: reporte lacuna,
dizendo o que tentou.

# 5. saída - o template do passo 6
```

Um pesquisador, **um assunto**. Cinco assuntos = cinco arquivos como este, ou **um** genérico invocado cinco vezes.

São independentes: rodam **todos ao mesmo tempo**. É daqui que vem o ganho de velocidade.

Escreva o consolidador

```
---  
name: consolidador  
tools: Read, Write, Glob, Grep ← sem web  
model: opus  
---
```

Você **não pesquisa**. Tudo vem dos arquivos de origem. Dado que faltar vira lacuna, não vira busca sua.

Se algum arquivo não existir,
pare e reporte quais faltam.

```
# regras de consolidação  
1. Renumere as fontes sem perder nenhuma.  
2. Conflito é achado, não erro: registre  
   os dois lados em ## Divergências.  
3. Célula sem respaldo fica vazia.
```

O consolidador **constata**, não julga. Decidir quem está certo é trabalho de outro agente, ou seu.

É opus porque a decisão é ambígua: o que é divergência real, o que é a mesma coisa dita com outra palavra.

Escreva o revisor

```
---
name: revisor
tools: Read, Glob, Grep ← nem Write
model: sonnet
---
```

Revisor *mecânico*. Não julga se o argumento é bom. Cada item é *PASSA* ou *FALHA*, com a evidência exata do que foi checado.

checklist

1. Todo [n] tem entrada em ## Fontes?
2. Nenhuma fonte se perdeu?
3. Toda célula tem respaldo?
4. Divergências preservadas?
5. Toda lacuna de origem aparece?

PASS: X/5 · FAIL: Y/5 | Rodada: N

ENCAMINHAMENTO: <uma de três>

O revisor **nunca edita** o que revisa, e o consolidador **nunca se auto-aprova**.

Três valores: **APROVADO**, **CONSOLIDADOR** com a lista de itens, ou **HUMANO**. Prosa não serve.

Rodar — as três fases

Fas Pesquisa, tudo em paralelo

e 1 Peça os N pesquisadores **numa mensagem só**. Um por vez faz eles rodarem em fila e você perde o ganho.

Fas Consolidação

e 2 Só depois que os N arquivos existirem. Confira com ls antes de chamar.

Fas Revisão

e 3 Chame o revisor e leia **uma linha**: a de encaminhamento.

Você não roda comando nenhum. Pede em português mesmo — *“use o agente pesquisador-editais”* — e o agente principal delega. O description do passo 4 é o que faz ele escolher certo.

O loop de correção

A linha diz	Você faz
APROVADO	segue para a próxima etapa do trabalho
CONSOLIDADOR – itens 2, 5	reinvoca o consolidador, que corrige só esses itens
HUMANO – motivo	o dado não existe nos arquivos-base: precisa de pesquisa nova

Contador de rodada

O revisor lê o número da rodada anterior e incrementa.

Teto de três

Na rodada 3 com falha, não há rodada 4: escala para você.

Anti-regressão

“Corrija só o apontado.” Reescrever o aprovado introduz erro novo.

Quando virar skill

Skill é **receita**, não agente. Serve quando a mesma sequência de passos se repete e você não quer redescobrir ela toda vez.

O sinal: você se pegou **colando o mesmo parágrafo** em três prompts. Isso não é regra de agente, é conhecimento — e conhecimento repetido deriva.

```
.claude/skills/  
└─ gerar-pdf/  
   └─ SKILL.md  
  
dentro do SKILL.md:  
---  
name: gerar-pdf  
description: Gera PDF a partir de  
             um .md finalizado. Use sempre  
             que for gerar PDF aqui.  
---  
  
1. Setup de ambiente (uma vez)  
2. Markdown → HTML+CSS  
3. HTML → PDF  
4. Conferir visualmente
```

ANTES DE SOLTAR VOCÊS

Os seis erros que vocês vão cometer

1. Dar busca ao revisor.

Ele conserta em vez de reprovar, e você nunca fica sabendo.

2. Pedir saída “bem estruturada”.

Sem o esqueleto colado no prompt, cada rodada volta diferente.

3. Achar que a regra escrita é trava.

É pedido. A trava é o settings.json.

4. Deixar o agente escrever em input/.

Perdeu o original, perdeu a chance de refazer.

5. Contrato preso a texto literal.

Meu revisor cresceu de 10 para 11 itens e o gatilho do consolidador continuou procurando a string antiga.

6. A mesma regra em quatro arquivos.

Muda em um só e deriva nos outros.

A mesma pasta, outros caminhos

```
meu-projeto/
├── AGENTS.md      a constituição
├── .agents/
│   ├── rules/     regras extras
│   ├── agents/<nome>.md
│   └── skills/<nome>/SKILL.md
├── input/   pdf/   scripts/
└── output/      iguais: são suas, não da ferramenta
```

As pastas de **dados** não mudam — são suas. Só muda onde a ferramenta procura a configuração.

```
global, vale em todo projeto:
~/.gemini/GEMINI.md
~/.gemini/config/agents/
~/.gemini/config/skills/
```

A regra pode morar em três lugares: AGENTS.md na raiz, vários arquivos em .agents/rules/, e o GEMINI.md global. Todos carregam juntos — o que é ótimo para separar, e péssimo se você repetir a mesma regra em dois deles.

O arquivo de agente, campo a campo

```
---
name: pesquisador-editais
description: ... ← o planner usa
tools: [lista explícita]
model: pro | flash | inherit
skills: [skills que ele enxerga]
subagent: true ← sem isto, ninguém chama
mainAgent: false ← porta de entrada?
permissionMode: ...
commandExecutionPolicy:
  off | auto | eager | sandbox
---
```

Mesmo formato: markdown com frontmatter YAML. As cinco seções do passo 5 e o template do passo 6 valem igual.

Quatro campos a mais, e dois deles resolvem coisas que no Claude Code eu não consigo:

skills limita quais receitas o agente enxerga. commandExecutionPolicy dá política de shell **por agente** — revisor em off, pesquisador em sandbox.

Invocação e permissões

Invocação — o agente pai chama pela ferramenta `invoke_subagent`: vários em paralelo, cada um com contexto limpo, até **10 níveis** de aninhamento.

Aninhar significa que o **orquestrador pode ser um agente**: alguém que lê a linha de encaminhamento, reinvoca o consolidador e conta as rodadas — em vez de você.

A grande diferença: existe `write_file(caminho)`. A regra de diretório do passo 1 deixa de ser texto e vira **trava de verdade** — dá para proteger `input/` no nível do sistema. No Claude Code eu só consigo barrar comandos.

Permissões — três níveis, precedência **deny > ask > allow**:

```
"deny": [ "command(rm -rf)",  
          "write_file(input/)" ],  
"allow": [ "command(python3 ./scripts/)" ],  
"ask":    [ "command(*)" ]
```

Três adaptações obrigatórias

01 Teto de 12.000 caracteres por arquivo de regra.

A minha constituição tem 10,2 mil — passa raspando. Se crescer, quebre em vários arquivos em `.agents/rules/` ou mova o glossário para uma skill.

02 **subagent: true** é explícito.

No Claude Code todo arquivo na pasta de agentes é invocável. Lá não: sem essa linha, o agente existe e nunca é chamado.

03 Os nomes das ferramentas são outros.

Não existe WebSearch ou Glob com esses nomes. Como o passo 7 depende de **omitir** a ferramenta certa, errar esse campo devolve busca ao revisor **em silêncio**.

Teste depois de migrar: peça ao revisor para buscar algo na web. Se ele conseguir, o campo tools está errado.

LEVE ISTO

Checklist de uma página

- ☐ seis pastas criadas
- ☐ input / pdf / output separados
- ☐ constituição escrita antes do 1º agente
- ☐ deny no arquivo de permissões
- ☐ cada agente cabe numa pergunta só
- ☐ esqueleto de saída colado no prompt
- ☐ toda tabela com colunas nomeadas
- ☐ ## Lacunas e ## Fontes em todo output
- ☐ revisor sem ferramenta de busca
- ☐ nomes de arquivo definidos antes dos prompts
- ☐ quem revisa não edita
- ☐ loop com contador e teto
- ☐ encaminhamento em formato fechado
- ☐ nenhuma regra escrita em dois lugares

Perguntas – e depois montamos a pasta de vocês juntos.